3D Space Partitioning Viewer

Computer Graphics (E016712A) – Project Report

Antonio Romero Martín Siebe Ternest

April 29, 2026

Contents

1 Introduction **3**

2 Reading the Input Data **3**

3 Space Partitioning Algorithms **3**
3.1 Octree 3
3.2 K-d Tree 4
3.3 BSP Tree 6

4 Viewer **7**
4.1 Rendering Architecture 7
4.2 Depth Navigation 8
4.3 Coloring 8
4.4 Wireframe 8
4.5 BSP Visualization 8
4.6 UI and Dataset Loading 9

5 Comparative Analysis **10**
5.1 Main Differences between the Three Trees 10
5.2 Benefits of an Octree over a BSP-Tree 10
5.3 Ray Intersection Queries 10
5.4 Dynamic Scenes 11
5.5 Urban Scenes 11

6 Conclusion **11**

1 Introduction

This report documents the design decisions, implementation challenges, and comparative analysis of a 3D space partitioning viewer built for LiDAR point clouds. Three hierarchical data structures were implemented: an **Octree**, a **k-d Tree**, and a **Binary Space Partitioning (BSP) Tree**.

Rather than raw OpenGL, the viewer is built with **Three.js** (WebGL). This let us focus on algorithm design and rendering concepts (scene graph, instanced geometry, depth testing, camera control) without low-level shader boilerplate. The project is structured as a single-page web application served by Vite, with all three algorithms living in a shared `src/algorithms/` module and all visualization logic in `src/viewer/`.

2 Reading the Input Data

(Both authors)

Input datasets are PCD (Point Cloud Data) files from LiDAR scanners. We implemented a custom PCD parser that handles both **ASCII** and **binary** encodings. The header is parsed line-by-line to extract the point count, field layout (`FIELDS`, `SIZE`, `TYPE`), and data format. For binary files a `DataView` reads the raw byte buffer directly into a `Float32Array`, which is handed to Three.js as a `BufferGeometry` position attribute.

The main challenge was that PCD files often include non-positional fields (intensity, normals, RGB). We handle this by computing per-point byte offsets from the `SIZE` header, extracting only the X, Y, Z channels and discarding the rest.

3 Space Partitioning Algorithms

3.1 Octree

(Both authors)

Design decisions

The tree is built top-down: starting from a bounding box around all points, it is recursively subdivided into 8 equal octants. A node stops splitting when either `maxDepth` is reached *or* its point count falls at or below `maxPointsPerNode` – whichever comes first.

The first design decision was to **force the root bounding box into a perfect cube** by extending the two shorter axes to match the longest one. This guarantees that *every voxel at a given depth has the same side length*, regardless of the dataset's spatial extent. The consequences are threefold: the leaf-size readout in the UI is meaningful because all leaves at the same depth are equally sized; ray-AABB intersection tests benefit from uniform cell dimensions that allow precomputed slab distances; and level-of-detail rendering is straightforward – querying a shallower depth uniformly coarsens the whole scene at the same rate in every direction.

A second decision was **pre-allocating eight point-bucket arrays per depth level** instead of creating them inside each recursive call. On a 200k point cloud, the naïve version allocated eight new arrays per internal node, generating significant garbage-collection pressure. With the pre-allocated table each bucket is reset to length zero and reused, keeping allocation overhead constant.

Only non-empty octants produce child nodes, the tree never stores empty nodes, so technically this implementation of the octree is a sparse octree (no always 8 childs). This is crucial for large, non-uniform point clouds, where many octants would be empty and waste memory and traversal time. It also improves the visibility of occupied regions in the viewer, since only nodes containing points are rendered.

Complexity and memory

- **Build:** $O(n \log n)$ – each point traverses $O(\log n)$ levels, with $O(1)$ work per level (the split plane is always the geometric midpoint, computed without examining the points).
- **Single-point update:** $O(\log n)$ – follow the path from root to leaf and add/remove the point from the appropriate voxel; the split planes never change.
- **Memory:** $O(n)$ – at most n leaf nodes at maximum depth; internal nodes free their point arrays once split.

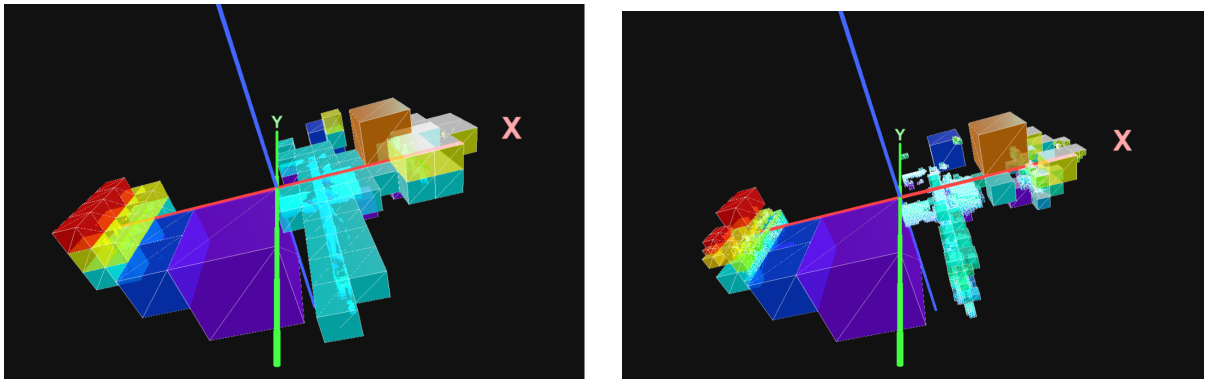

Figure 1: Octree on the Corridor dataset at a low depth (left) and a high depth (right). Color encodes the Z coordinate (blue = low, red = high). The wireframe outlines show voxel boundaries.

3.2 K-d Tree

(Both authors)

Design decisions

The split plane is always placed at the **median** of the active coordinate. Using the median rather than the geometric midpoint is the key choice: it guarantees that each split divides the point set into two roughly equal halves regardless of point density, keeping the tree balanced.

Axis-selection strategies. Three strategies are implemented and selectable at run time via the **Split Mode** control in the viewer:

- **Cycle** (default): the axis rotates X → Y → Z with depth (`depth % 3`). Simple and data-independent; all nodes at the same depth always split on the same axis.

- **Widest**: picks the axis whose bounding-box extent is largest at the current node, using the tight AABB already computed during construction. $O(1)$ per node; ensures the longest dimension is always halved first, producing more cube-like cells and reducing long thin partitions.

- **Variance**: picks the axis with the highest variance of the point coordinates – the dominant diagonal entry of the covariance matrix, equivalent to what a full PCA would identify as the principal axis of spread when restricted to axis-aligned splits. Variance is estimated via **Welford's online algorithm** on a stride sample of up to 1 024 points, keeping per-node cost $O(1)$ in practice while remaining numerically stable for coordinates of any scale. Unlike *widest*, this strategy responds to the actual distribution of points rather than just the bounding geometry.

A module-level scratch `Vector3` is reused across all *widest* computations to avoid per-call heap allocations.

Median selection. Finding the median without a full sort is done with a **3-way quickselect** (Dutch National Flag partitioning). This is $O(n)$ average per node and handles duplicate coordinates correctly by grouping equal-valued points into a pivot zone in a single pass, so repeated coordinates in the cloud do not degrade performance to $O(n^2)$ as they would with a naïve quicksort.

All points are stored in a **single shared array** that is partitioned in-place during construction. Each node records a `[start, end)` range into this array – no per-node point copies are made. Bounding boxes are computed from the actual point distribution after partitioning, giving tight cuboids instead of the oversized AABB of the raw split range.

Balancing

Because every split is at the median, the k-d tree is **structurally balanced**: each partition divides the point set into two equal-count halves, bounding the tree depth at $\lceil \log_2 n \rceil$. In practice, however, the `maxPointsPerNode` leaf threshold means that a sparse region can become a leaf several levels earlier than a dense region that keeps splitting. The tree therefore remains **well-balanced** – traversal depth stays close to $O(\log n)$ – but the strict guarantee that all leaves sit at the same depth does not hold when the leaf threshold is active.

A degenerate edge case – all points sharing the same coordinate on the split axis – is caught explicitly: after quickselect, if the left partition is empty the node is finalized as a leaf immediately rather than recursing into a zero-progress split.

A balanced tree is especially important for ray traversal: in a balanced tree the worst-case traversal depth is $O(\log n)$, whereas an unbalanced tree can degrade to $O(n)$.

Complexity and memory

- **Build:** $O(n \log n)$ – quickselect is $O(n)$ per node; there are $O(\log n)$ levels; total work per level is $O(n)$.

- **Query / ray intersection:** $O(\log n)$ average due to balance.

- **Single-point update:** $O(\log n)$ for search; however, naïve insertion without rebalancing degrades the tree over time.
- **Memory:** $O(n)$ during construction; $O(\log n)$ per cached tree thereafter. The shared point array (n `Vector3` objects) is freed immediately after the tree is fully built, so multiple cached variants (Cycle, Widest, Variance) for the same dataset can coexist in memory without each retaining a full copy of the point cloud.

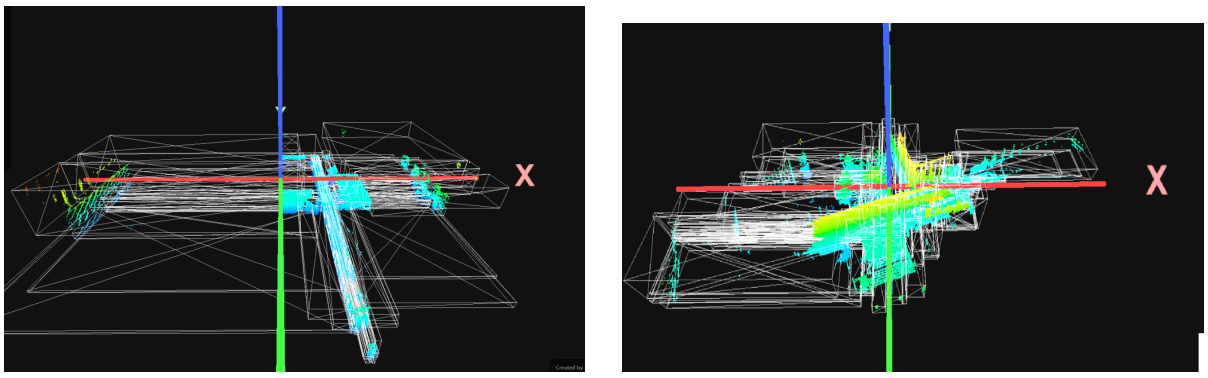

Figure 2: K-d Tree on the Corridor dataset (left) and the UFO dataset (right). The cuboids are non-uniform because the median split follows the actual point distribution, not a fixed midpoint.

3.3 BSP Tree

(Antonio Romero Martín)

Design decisions

A BSP tree splits space with **arbitrarily oriented planes**. The central design challenge is choosing a good plane at each node. We evaluated three strategies:

1. **Random plane** – fast but produces uneven splits.
2. **Axis-aligned median** – equivalent to k-d tree; does not exploit non-axis-aligned structure. PCA: Principal component analysis.
3. **PCA-based plane** – the plane normal is the dominant eigenvector of the covariance matrix of the node's points; the plane passes through the centroid.

We chose PCA because it adapts to the local geometry: in a corridor the principal axis aligns along the corridor, producing long thin cells; in an open area the plane follows the dominant spread of points. This reduces the likelihood of degenerate splits compared to random or axis-aligned strategies.

The eigenvector is found by **power iteration** (up to 20 iterations on the 3×3 covariance matrix, with early termination when the vector magnitude falls below 10^{-10}), which converges reliably for real-world point clouds. Coordinates are copied into three pre-allocated `Float32Array` buffers for cache-friendly covariance computation, avoiding per-node allocations inside the hot loop.

A node stops splitting when either `maxDepth` is reached *or* its point count falls at or below `maxPointsPerNode`. If the PCA plane produces a degenerate partition – all points falling on the same side – the node is finalized as a leaf immediately to avoid infinite recursion on numerically degenerate inputs.

Why BSP takes longer to build than an Octree

An octree's split plane is always the three axis-aligned midplanes of the current node – computed in $O(1)$ regardless of point count. A BSP node must **examine all its points** to compute the mean, the covariance matrix, and run power iteration before any child can be created: that is $O(n_{\text{node}})$ per node. Summed across all nodes the asymptotic build cost is the same ($O(n \log n)$), but the constant factor is much larger. On the Hasselt dataset ($\sim$14 million points, 225 MB) we measured BSP taking roughly 3$\times$ longer than the Octree.

Complexity and memory

- **Build:** $O(n \log n)$ asymptotically, but with a large constant due to PCA.
- **Single-point update:** $O(\log n)$ to locate the correct leaf, but insertion is expensive in practice: adding a point can shift the local PCA plane, potentially invalidating the split for that node and requiring a rebuild of the affected subtree.
- **Memory:** $O(n)$ – leaf nodes store a compact `Int32Array` of geometry indices; full point arrays are freed after splitting each node.

4 Viewer

4.1 Rendering Architecture

(Both authors)

The viewer uses a `PerspectiveCamera` (75° FOV) with `OrbitControls` for mouse-driven orbit. Pan and right-click are disabled to keep navigation simple; rotation damping (factor 0.08) smooths the motion without overshoot. On each dataset load the camera is automatically repositioned via a bounding-sphere fit: the distance is set proportionally to the sphere radius so the entire point cloud fills the viewport.

Two services decouple concerns cleanly. `datasetService` owns point cloud loading, height-ramp colorization, and camera fitting. `partitionService` owns tree construction, per-variant caching, and depth queries. Trees are built **on first use** and stored under a composite key `algorithm::dataset::variant`. Switching algorithms, changing depth, or toggling between the k-d tree's three split-mode variants is instant once the corresponding tree is cached – no rebuild is triggered on subsequent interactions.

For Octree and k-d Tree, active nodes at the current depth are rendered as **two `InstancedMesh` objects**: a semi-transparent solid box mesh and a wireframe overlay, both backed by a single shared `BoxGeometry`. Using `InstancedMesh` instead of one `Mesh` per node reduces draw calls from thousands to one, which proved essential: without instancing, rendering depth-8 Octree nodes ($\sim$8000 boxes) dropped the frame rate below 10 fps; with instancing it stayed above 60 fps.

4.2 Depth Navigation

(Both authors)

The `+` and `-` keys change the display depth. The tree is queried with `getNodesAtDepth(d)`: nodes exactly at depth d are included, and leaves shallower than d are also included so coarser resolutions still show all occupied regions. This traversal is O(active nodes) per key press, while the expensive build runs only once.

4.3 Coloring

(Both authors)

Octree cells are colored by the Z coordinate of their bounding-box center using a **height ramp**: HSL hue is linearly mapped from 0.7 (blue/purple, low Z) to 0.0 (red, high Z). This makes the vertical structure of urban point clouds immediately visible and matches the conventional LiDAR viewer palette.

K-d Tree cells are colored by the split axis that created them – that is, the axis of the parent split that produced the boundary between a node and its sibling. The three colors match the on-screen XYZ axis helper exactly: red for X, green for Y, and **blue** for Z. Because both children of a split receive the same axis tag, sibling cells always share the same hue, making the direction of each partition boundary immediately legible without any additional annotation. With the *Cycle* mode all cells at a given depth have the same color; with *Widest* and *Variance* the colors at a given depth can be mixed, directly reflecting the data-driven axis choices made at each node.

4.4 Wireframe

(Both authors)

Each active node is rendered with a second `InstancedMesh` using `wireframe: true`, making voxel boundaries visible without obscuring the interior color. Solid fill and wireframe can be toggled independently via checkboxes in the control panel.

4.5 BSP Visualization

(Antonio Romero Martín)

BSP cells are convex polytopes bounded by arbitrary planes, so axis-aligned boxes are not a valid representation. We chose to **recolor the point cloud in place**: each leaf cell is assigned a distinct hue by cycling evenly across the HSL color wheel. This makes cell boundaries visible directly in the data without needing to compute convex hulls (which would be expensive and visually complex for thousands of cells).

Recoloring operates at the GPU buffer level. During construction each BSP leaf records a compact `Int32Array` of the original geometry indices of its points. When BSP mode is activated, the visualizer saves a full copy of the Three.js `BufferGeometry` color attribute (the height-ramp colors applied at load time) and then writes each leaf's HSL color directly into the buffer at the stored indices. When the user switches away from BSP mode, the saved copy is written back, restoring the baseline appearance without reloading the dataset.

Additionally, the **split planes** are shown as semi-transparent quads oriented to the PCA normal at each internal node up to the current depth (Figure 3). Each quad is positioned at the split centroid, scaled to the node's bounding-box size, and oriented via

quaternion rotation from the world up-vector to the plane normal. This gives an intuitive view of how space is partitioned level by level.

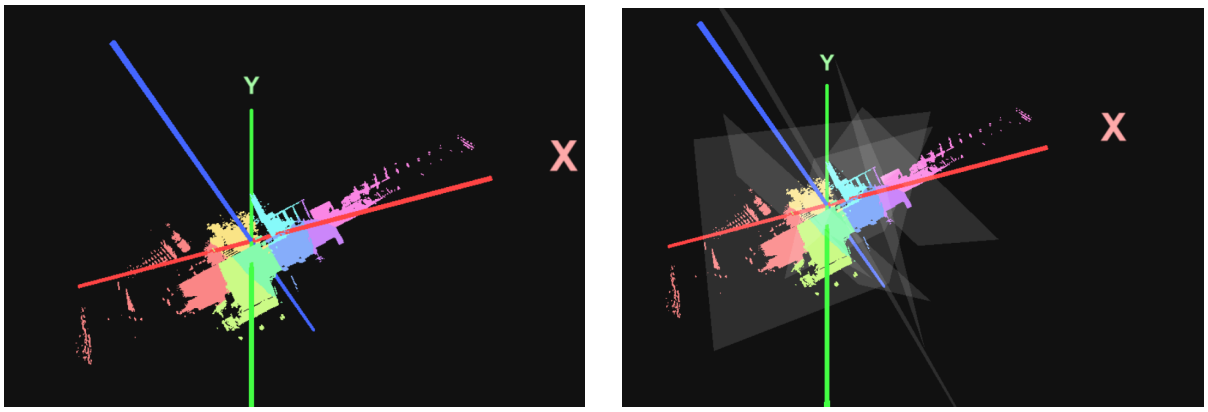

Figure 3: BSP Tree: point cloud colored per leaf cell (left), and split planes shown as semi-transparent quads (right).

4.6 UI and Dataset Loading

(Siebe Ternest)

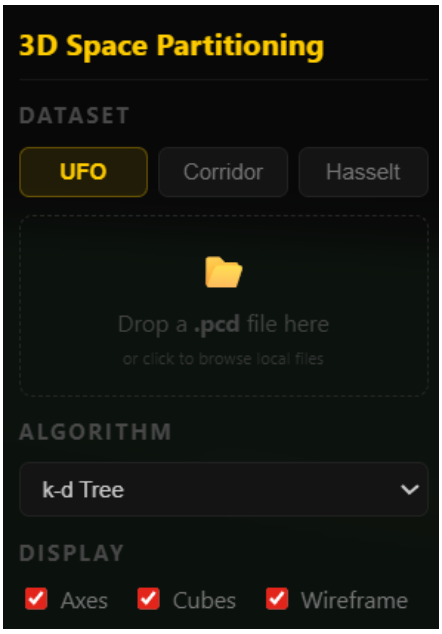

Figure 4: Control panel with algorithm selector, dataset buttons, depth display, leaf-size readout, and visibility toggles.

The control panel (Figure 4) provides algorithm and dataset selection, solid/wireframe visibility toggles, and an XYZ axes toggle. Depth and average leaf size are updated in real time. The **leaf size** readout displays the mean cube root of active-node volumes, $\bar{s} = \frac{1}{N}\sum_{i=1}^{N} \sqrt[3]{V_i}$, giving an intuitive single-number summary of the current partitioning resolution that is comparable across algorithms and datasets.

In addition to the three bundled datasets (UFO, Corridor, Hasselt), users can drag and drop any `.pcd` file onto the viewer. The application runs entirely in the browser;

datasets are fetched over HTTP or read from the local file system via the File API.

When the k-d Tree algorithm is selected, a **Split Mode** selector appears in the control panel offering the three axis-selection strategies (Cycle, Widest, Variance). Each variant is cached independently under a composite key `algorithm::dataset::mode`, so switching between modes after the first build is instant. The active mode is highlighted with the same yellow accent used for dataset buttons, making the current strategy immediately visible.

5 Comparative Analysis

5.1 Main Differences between the Three Trees

	Octree	**K-d Tree**	**BSP Tree**
Split planes per node	3 axis-aligned	1 axis-aligned	1 arbitrary
Children per node	Up to 8	Exactly 2	Exactly 2
Split position	Fixed midpoint	Data median	PCA centroid
Guaranteed balanced	No	Yes (in practice)	No
Build cost (constant)	Low	Medium	High
Cell shape	Cube	Cuboid	Convex polytope

The Octree always cuts at the geometric midpoint along all three axes at once, giving $O(1)$ split decisions. The k-d tree cuts one axis at a time at the median, making it self-balancing at the cost of a linear scan. The BSP tree uses an arbitrary plane adapted to the local point geometry, which produces the most informative partitions but incurs the highest build cost.

5.2 Benefits of an Octree over a BSP-Tree

- **Simpler, faster construction:** No covariance computation; the split planes are determined in $O(1)$.
- **Easy updates:** Inserting or removing a point only affects the nodes on its root-to-leaf path; split planes never change, so no structural rebuild is needed.
- **Predictable cell shapes:** All cells at a given depth are cubes of identical size, simplifying ray-AABB tests and frustum culling.
- **Implicit empty-space compression:** Only occupied octants create child nodes, making the octree naturally sparse. Also since we implemented a sparse octree we don't create 8 child nodes for every internal node, but only for the non-empty octants. This is a significant advantage in large, non-uniform point clouds where many octants would be empty and waste memory and traversal time.

5.3 Ray Intersection Queries

All three trees support $O(\log n)$ average ray traversal by pruning subtrees whose bounding volumes do not intersect the ray. In practice:

- **Octree:** Ray-AABB intersection (slab method) is very fast with six comparisons per node; the regular grid at each level enables front-to-back ordering, allowing early termination once the first hit is found.
- **K-d Tree:** Each internal node test reduces to a single axis comparison (dot product with an axis vector), making it the cheapest per-node test. Balance guarantees $O(\log n)$ depth.
- **BSP Tree:** Each node requires a full dot product with an arbitrary normal, which is slightly more expensive. The tree is not guaranteed to be balanced, which can increase traversal depth beyond $O(\log n)$ in the worst case.

For ray casting, the **k-d tree** is generally the best choice: its balance guarantee minimizes tree depth, and axis-aligned planes minimize per-node cost.

5.4 Dynamic Scenes

For dynamic scenes with moving objects or streaming LiDAR, we would use an **Octree**. Its regular structure makes insertion and deletion local operations: a moving point only modifies the nodes on its root-to-leaf path, and split planes never need updating. Unlike the k-d tree, which typically requires periodic full rebuilds to restore balance after many insertions, the octree supports truly incremental updates with no rebalancing step. The BSP tree's data-dependent planes make any update expensive since the PCA result changes when the point distribution shifts, potentially invalidating the split planes of entire subtrees.

5.5 Urban Scenes

For urban scenes we would use an **Octree**. While the k-d tree is also a valid choice – its median split adapts well to non-uniform point densities – the octree has two practical advantages that matter specifically for urban LiDAR data.

First, urban point clouds are extremely large (millions of points) and spatially non-uniform: very dense near building facades and the ground plane, almost empty in open air. A sparse octree compresses this empty space implicitly and at multiple resolutions simultaneously, making it straightforward to implement level-of-detail rendering by simply querying a shallower depth. The k-d tree does not have a natural notion of uniform resolution levels, since its cuboids vary in size and shape across the tree.

Second, urban mapping applications typically require repeated spatial queries of the same type (e.g. nearest-neighbour search, frustum culling, occupancy checks), all of which benefit from the octree's predictable, regular cell structure. Practical LiDAR mapping tools such as OctoMap confirm this choice as the industry standard for large-scale outdoor scenes.

6 Conclusion

We implemented three spatial partitioning structures – Octree, k-d Tree, and BSP Tree – and integrated them into a single interactive browser-based viewer for LiDAR point clouds, built with Three.js rather than raw OpenGL. This allowed us to focus on algorithm design

and rendering architecture without low-level graphics boilerplate, at no cost to visual quality or interactivity.

The comparative analysis confirmed that no single tree dominates across all criteria. The **octree** is the most practical general-purpose structure: fast to build, easy to update incrementally, and naturally suited to level-of-detail rendering of large urban scenes. The **k-d tree** is the best choice for ray casting and nearest-neighbour queries, where its balance guarantee and cheap per-node axis test minimize traversal cost. The **BSP tree** produces the most geometry-adaptive partitions, but its PCA-based build cost and lack of balance make it less suitable for dynamic or real-time applications.

On the implementation side, two decisions had the largest practical impact. **InstancedMesh rendering** reduced thousands of draw calls to one, keeping the frame rate above 60 fps at deep tree levels where per-node meshes failed entirely. **Tree caching** – building each tree on first use and storing it under a composite (algorithm, dataset, variant) key so that depth navigation and algorithm or split-mode switching require no rebuild – was the single most impactful UX decision, making all interactions instant even on the largest tested dataset (Hasselt, $\sim$14 million points).

Overall, the project reinforced that the choice of spatial data structure is inseparable from the intended query type, update frequency, and scene characteristics – a lesson that extends well beyond point cloud processing to any geometry-heavy application.